
Mamba Biometric EKG Analysis Technology (MambaBEAT)

**Sergio Rodríguez Vidal Álvaro Pereira Chagoyen Antonio Mora Abós
Jaime Paz Rodríguez**

Comillas Pontifical University ICAI
sergio.rodriguez.vidal@sap.com
{pere03, antoniomoraabos}@alu.comillas.edu
jaime.pazrodriguez@zurich.com

Abstract

The field of Electrocardiogram (EKG/ECG) analysis has seen significant advancements with the advent of deep learning models. State-of-the-art models such as ResNet, Transformer, ViT, and H3 have demonstrated remarkable performance in EKG classification tasks. However, the quest for more efficient and accurate models is a continuous journey. This proposal introduces a novel project, provisionally named Mamba Biometric EKG Analysis Technology (MambaBEAT), which aims to leverage the Mamba model, a recent and promising sequence modeling architecture, for EKG analysis.

1 Introduction

NO MANTENER INTRODUCCION !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

In recent years, deep learning models have revolutionized the field of sequence modeling, with architectures such as Transformers demonstrating remarkable performance across various modalities including language, audio, and genomics. However, despite their success, these models often suffer from computational inefficiencies, particularly when dealing with long sequences.

Enter Mamba, a novel sequence modeling architecture that addresses these challenges. Mamba boasts a 5× higher throughput than Transformers and exhibits linear scaling in sequence length, making it particularly suited for handling real data up to million-length sequences. Furthermore, Mamba has been shown to outperform Transformers of the same size and even match the performance of Transformers twice its size, both in pretraining and downstream evaluation.

Given these advantages, we chose to leverage the Mamba model for our project, Mamba Biometric EKG Analysis Technology (MambaBEAT). The field of Electrocardiogram (EKG/ECG) analysis, which involves the interpretation of long sequences of heart rhythm data, stands to benefit significantly from Mamba's efficiency and scalability. Our proposal aims to harness the power of Mamba to drive advancements in EKG classification tasks, pushing the boundaries of what is currently achievable with state-of-the-art models.

2 State Space Models

Traditional state space models, or SSM, are a mathematical framework used to describe the dynamics of a continuous system. They form the foundation of the Mamba model, which is the focus of this paper.

SSMs are defined by a simple equation. This equation takes a 1-D input signal, $x(t)$, and maps it to an N-D latent state, $h(t)$. It then projects this state to a 1-D output signal, $y(t)$.

The equation is as follows:

$$\begin{aligned} h'(t) &= Ah(t) + Bx(t) \\ y(t) &= Ch(t) + Dx(t) \end{aligned} \quad (1)$$

In a deep sequence model, we use the SSM as a black-box representation. The parameters A , B , C , D are learned through gradient descent. For the rest of our discussion, we'll leave out the parameter D in Equation 1. We can do this because the term Dx can be seen as a skip connection, which is easy to compute.

2.1 Discretization

In practical scenarios, continuous measurement or control of systems is not feasible, despite the inherently continuous nature of reality. This renders the handling of continuous derivatives challenging.

The transition from a continuous SSM to a discrete SSM:

$$\begin{aligned} h_t &= \bar{A}h_{t-1} + \bar{B}x_t \\ y_t &= Ch_t \end{aligned} \quad (2)$$

can be achieved through a series of mathematical transformations (e.g. discretization rules), as shown in Equations 7 to 14. These transformations lead to the Zero-Order Hold (ZOH) Discretization Rule (Equation 3) and the Bilinear Discretization Rule (Equation 4), which are fundamental to the operation of **Structured state space models (S4)**.

$$\bar{A} = \exp(A\Delta), \quad \bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I)\Delta B \quad (3)$$

$$\bar{A} = (I - \frac{1}{2}\Delta A)^{-1}(I + \frac{1}{2}\Delta A), \quad \bar{B} = (I - \frac{1}{2}\Delta A)^{-1}\Delta B \quad (4)$$

Equation 2 is a **sequence-to-sequence map** instead of function-to-function. In this equation, the state at time t , h_t , depends only on the previous time step, h_{t-1} , and the input at time t , x_t . This property is known as the **Markov property**, which states that the future state depends only on the current state and not on the sequence of events that preceded it.

In addition to the Markov property, Equation 2 also exhibits the **Linear Time-Invariance (LTI)**. An LTI system is characterized by the principle of superposition, which means that the response to a sum of inputs is the sum of the responses to each input individually. Furthermore, the system is time-invariant, meaning that whether an input is applied now or at some future time, the output will be the same, just shifted in time. In other words (Δ , A , B , C), and consequently ($\bar{A}$, $\bar{B}$) as well, are fixed for all time-steps.

2.2 Computation

After the discretization of the parameter matrices (Δ , A , B , C) into ($\bar{A}$, $\bar{B}$), the S4 model can be computed in two primary ways: **linear recurrence** and **convolution**.

1. **Linear Recurrence:** In this method, the computation of the model is performed iteratively, using Equation 2. This is a sequential process, where each step depends on the previous one. While this method can be computationally intensive, it is straightforward and easy to implement.
2. **Convolution:** This method is more commonly used because it allows for parallelization in training, which can significantly speed up the computation process. The Convolution method involves the application of a function or a kernel over the input data. This allows Equation 2 to be rewritten as Equation 5:

$$\begin{aligned} \bar{K} &= \{C\bar{B}, C\bar{A}\bar{B}, \dots, C\bar{A}^k\bar{B}, \dots\} \\ y(t) &= \bar{K} * x(t) \end{aligned} \quad (5)$$

where $y(t)$ can be rapidly computed using Fast Fourier Transform thanks to the Convolution theorem.

2.3 Long-Range Dependencies with HiPPO

Structured SSMs are so named because computing them efficiently also requires imposing structure on the A matrix. Prior models suggest a special initialization based on the HiPPO theory.

HiPPO is a framework for compressing continuous and discrete data projecting them onto a polynomial basis, which means the polynomial coefficients will be stored in the hidden state[3].

Most used are **Legendre Polynomials** scaled through a window over time (to avoid forgetting), resulting on the **HiPPO-LegS** matrix, defined element-wise as:

$$A_{nk} = \begin{cases} \sqrt{(2n+1)}\sqrt{(2k+1)} & \text{if } n > k \\ n+1 & \text{if } n = k \\ 0 & \text{if } n < k \end{cases} \quad (6)$$

This matrix has quite interesting properties: invariant to timescale, fast to compute and has **bounded gradients** (avoids the vanishing/exploding gradients problem).

In order to speed up matrix A operations, it can be decomposed as a *Normal Plus Low-Rank* matrix (NPLR)[2]

3 Selective State Space Models

While S4 Models have shown to be more computationally efficient and capable of handling long-range dependencies better than some state-of-the-art models like the Transformer, they do have their limitations.

One of the key challenges with S4 Models is their performance in context-dependent tasks. This is partly due to the Linear Time Invariance (LTI) property of S4 Models, which makes the input-dependence seen in the attention mechanism of Transformers impossible. The attention mechanism in Transformers allows the model to focus on different parts of the input sequence when producing an output, thereby capturing the context more effectively. This level of context sensitivity is something that S4 Models, due to their LTI property, struggle to achieve.

To address this limitation, Albert Gu and Tri Dao introduced **Selective State Space Models (SSMs)** in their Mamba model. Selective SSMs allow the parameter matrices (Δ, B, C) be input dependent. o be dependent on the input. Specifically, for an input $x : (B, L, D)$, they define:

- $B : (B, L, N) \leftarrow \text{Linear}_N(x)$
- $C : (B, L, N) \leftarrow \text{Linear}_N(x)$
- $\Delta : (B, L, D) \leftarrow \text{Broadcast}_D(\text{Linear}_N(x))$.

3.1 Parallelization

With the introduction of dynamic parameters in Mamba, the precomputation of the convolution kernel becomes unfeasible. However, to circumvent this challenge and enable parallelization, Mamba incorporates three classical techniques: kernel fusion, parallel scan, and recomputation.

References

- [1] Mamba: Linear-Time Sequence Modeling with Selective State Spaces. [Online]. Available: <https://arxiv.org/abs/2312.00752>
- [2] Efficiently Modeling Long Sequences with Structured State Spaces. [Online]. Available: <https://arxiv.org/pdf/2111.00396.pdf>
- [3] HiPPO: Recurrent Memory with Optimal Polynomial Projections. [Online]. Available: <https://arxiv.org/abs/2008.07669>

- [4] How to Train Your HiPPO: State Space Models with Generalized Orthogonal Basis Projections. [Online]. Available: <https://arxiv.org/abs/2206.12037>
- [5] Combining Recurrent, Convolutional, and Continuous-time Models with Linear State-Space Layers. [Online]. Available: <https://arxiv.org/abs/2110.13985>
- [6] MambaByte: Token-free Selective State Space Model. [Online]. Available: <https://arxiv.org/abs/2401.13660>
- [7] Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model. [Online]. Available: <https://arxiv.org/abs/2401.09417>
- [8] Attention Is All You Need. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [9] Neural Machine Translation by Jointly Learning to Align and Translate. [Online]. Available: <https://arxiv.org/abs/1409.0473>
- [10] HeartBEiT: Vision Transformer for Electrocardiogram Data Improves Diagnostic Performance at Low Sample Sizes. [Online]. Available: <https://arxiv.org/abs/2212.14040>
- [11] GPU Gems 3 by Hubert Nguyen. [Online]. Available: <https://developer.nvidia.com/gpugems/gpugems3>

A Discretization Rules

To transition from a continuous to a discrete model, the derivative in the State Equation 1 must be eliminated. This can be achieved by integrating both sides of the equation. However, before integration, both sides of the equation are multiplied by an exponential matrix, as shown in Equation 7:

$$e^{-At}h'(t) = e^{-At}Ah(t) + e^{-At}Bx(t) \quad (7)$$

This manipulation is a clever application of the product rule from calculus, where $u = h(t)$ and $v = e^{-At}$. Applying this rule backwards results in Equation 8:

$$\frac{d}{dt}[e^{-At}h(t)] = e^{-At}Bx(t) \quad (8)$$

Now, both sides of the equation can be integrated more easily, as shown in Equation 9:

$$\begin{aligned} \int_0^t \frac{d}{dt}[e^{-A\tau}h(\tau)]d\tau &= \int_0^t e^{-A\tau}Bx(\tau)d\tau \\ e^{-At}h(t) - h(0) &= \int_0^t e^{-A\tau}Bx(\tau)d\tau \end{aligned} \quad (9)$$

Multiplying by e^{At} and moving the constant term $h(0)$ to the right side leads to the following equation:

$$h(t) = e^{At}h(0) + \int_0^t e^{A(t-\tau)}Bx(\tau)d\tau \quad (10)$$

The next step is to define the discrete state $h_k = h(k\Delta)$ and plug it into the analytical solution, which results in Equation 11:

$$h_k = e^{Ak\Delta}h(0) + \int_0^{k\Delta} e^{A(k\Delta-\tau)}Bx(\tau)d\tau \quad (11)$$

It is now easy to predict the next state based on the previous one, as shown in Equation 12:

$$h_{k+1} = e^{A\Delta}h_k + \int_{k\Delta}^{(k+1)\Delta} e^{A((k+1)\Delta-\tau)}Bx(\tau)d\tau \quad (12)$$

The only remaining task now is to solve the integral. Let $u(t) = (k+1)\Delta - \tau$ and $dv = -d\tau$:

$$\begin{aligned} h_{k+1} &= e^{A\Delta}h_k + \left(\int_0^\Delta e^{Av}dv \right) Bx_k \\ h_{k+1} &= e^{A\Delta}h_k + A^{-1}(e^{A\Delta} - I)Bx_k \end{aligned} \quad (13)$$

A.1 Zero-Order Hold (ZOH) Discretization Rule

From Equation 13, we obtained the Zero-Order Hold (ZOH) Discretization Rule:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t \quad (2)$$

where $\bar{A} = \exp(A\Delta)$ and $\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I)\Delta B$ (Equation 3).

A.2 Bilinear Discretization Rule

To obtain the Bilinear Discretization Rule (Equation 4), the Forward Euler Approximation $e^{A\Delta} \approx I + A\Delta$ (First order Taylor Expansion) and Backward Euler Approximation $e^{A\Delta} \approx (I - A\Delta)^{-1}$ are used.

$$\bar{A} = \exp(A\Delta) = (\exp(A\Delta))^\alpha (\exp(A\Delta))^{1-\alpha} = (I - \alpha A\Delta)^{-1} (I + (1 - \alpha)A\Delta) \quad (14)$$

This family of α -dependent discretization methods on a parameter is called the **Generalized Bilinear Transform (GBT)**. If $\alpha = 0$, then it becomes the **classic Euler method** which is simply the first-order approximation $x(t + \Delta t) = x(t) + \Delta t \cdot x(t)$. If $\alpha = 1$ then it is called the **backward Euler**.

When $\alpha = 0.5$ then the **Bilinear Discretization Rule** is obtained (which preserves the stability of the system):

$$\bar{A} = (I - \frac{1}{2}\Delta A)^{-1}(I + \frac{1}{2}\Delta A), \quad \bar{B} = (I - \frac{1}{2}\Delta A)^{-1}\Delta B \quad (4)$$